

Week 2 Module:

Date: 17/06/2026

1. Find Student Roll Number (Linear Search)

Problem Statement

Your class teacher has a list of student roll numbers stored in the order they registered. Since the list is unsorted, you need to find whether a particular student roll number exists.

Write a program to search for the given roll number using **Linear Search**.

Input Format

- First line contains integer N (number of students)
- Second line contains N roll numbers
- Third line contains integer X (roll number to search)

Output Format

- Print "Found at index i" if present
- Otherwise print "Not Found"

Constraints

- $1 \leq N \leq 1000$

Example

Input

```
5
101 105 113 110 130
110
```

Output

```
Found at index 3
```

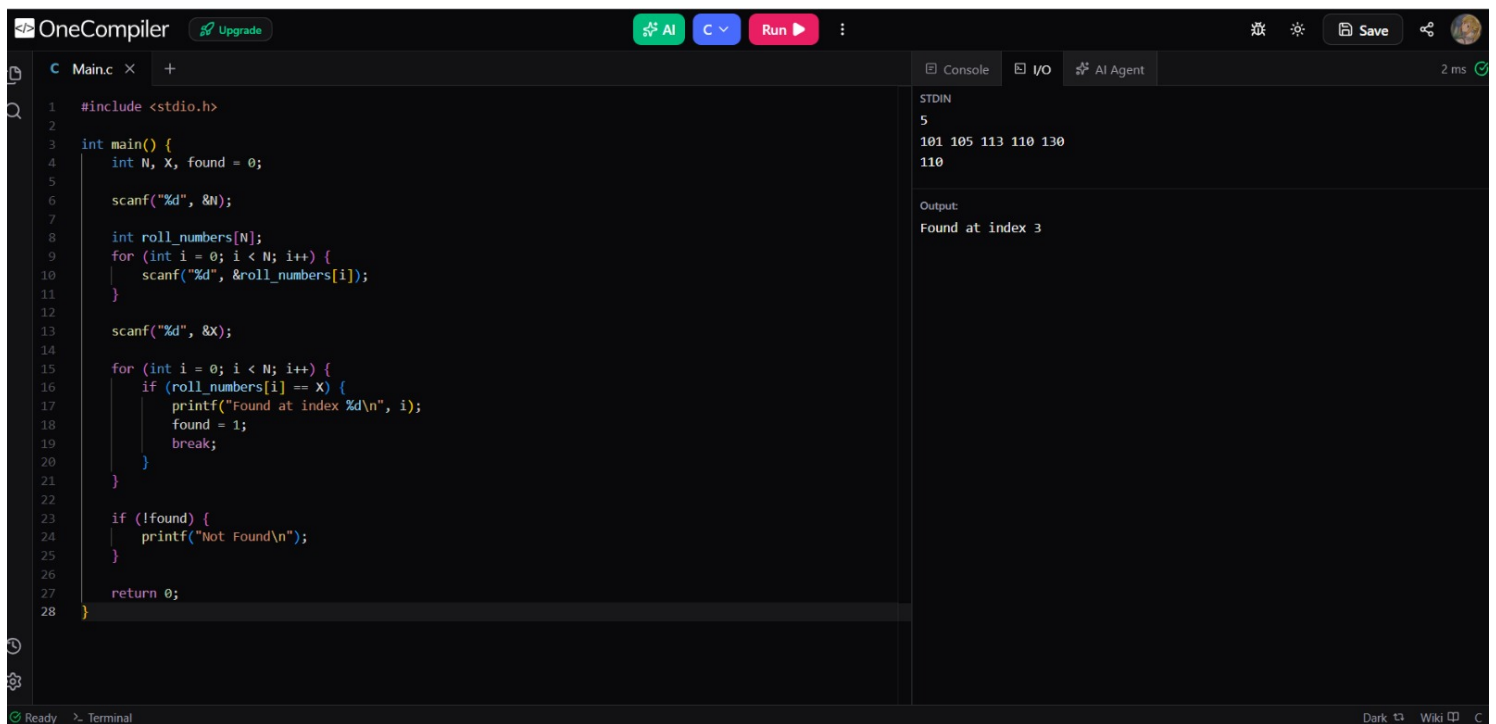
1 ⇒ Find student roll number (Linear Search)

```
#include <stdio.h>
int main() {
    int N, X, found = 0;
    scanf ("%d", &N);
    int roll-numbers[N];
    for (int i = 0; i < N; i++) {
        scanf ("%d", &roll-numbers[i]);
    }
    scanf ("%d", &X);
    for (int i = 0; i < N; i++) {
        if (roll-numbers[i] == X) {
            printf ("Found at index %d\n", i);
            found = 1;
            break;
        }
    }
    if (!found) {
        printf ("Not found\n");
    }
    return 0;
}
```

Input : 5

101 105 113 110 130
110

Output : Found at index 3



2. Search Product ID (Binary Search)

Problem Statement

An online store keeps product IDs sorted in ascending order. A customer wants to know whether a particular product exists in inventory.

Write a program to search the product using **Binary Search**.

Input Format

- First line contains N
- Second line contains sorted product IDs
- Third line contains target ID

Output Format

Print:

Product Available

or

Product Not Available

Constraints

- $1 \leq N \leq 10^5$

Example

Input

```
6
10 20 30 40 50 60
40
```

Output

```
Product Available
```

2 ⇒ Search Product ID (Binary Search)

```
#include <stdio.h>
```

```
int main() {
```

```
int N, target, found = 0;
```

```
scanf("%d", &N);
```

```
int product_ids[N];
```

```
for (int i = 0; i < N; i++) {
```

```
    scanf("%d", &product_ids[i]);
```

```
}
```

```
scanf("%d", &target);
```

```
int low = 0;
```

```
int high = N - 1;
```

```
while (low <= high) {
```

```
    int mid = low + (high - low) / 2;
```

```
    if (product_ids[mid] == target) {
```

```
        printf("Product Available\n");
```

```
        found = 1;
```

```
        break;
```

```
    } else if (product_ids[mid] < target) {
```

```
        low = mid + 1;
```

```
    } else {
```

```
        high = mid - 1;
```

```
    }
```

```
}
```

```
if (!found) {
```

```
    printf("Product Not Available\n");
```

```
}
```

```
return 0;
```

```
}
```

Input : 6

10 20 30 40 50 60

40

Output : Product
Available

OneCompiler

Upgrade

AI

C

Run

Main.c

+

```
1 #include <stdio.h>
2 int main() {
3     int N, target, found = 0;
4     scanf("%d", &N);
5     int product_ids[N];
6     for (int i = 0; i < N; i++) {
7         scanf("%d", &product_ids[i]);
8     }
9     scanf("%d", &target);
10    int low = 0;
11    int high = N - 1;
12    while (low <= high) {
13        int mid = low + (high - low) / 2;
14        if (product_ids[mid] == target) {
15            printf("Product Available\n");
16            found = 1;
17            break;
18        } else if (product_ids[mid] < target) {
19            low = mid + 1;
20        } else {
21            high = mid - 1;
22        }
23    }
24    if (!found) {
25        printf("Product Not Available\n");
26    }
27    return 0;
28 }
```

Console

I/O

AI Agent

2 ms

STDIN

6

10 20 30 40 50 60

40

Output

Product Available

Ready

Terminal

Dark

Wiki

C

3. Arrange Exam Scores (Bubble Sort)

Problem Statement

A teacher wants to arrange students' exam scores in ascending order to prepare the result sheet.

Write a program using **Bubble Sort**.

Input Format

- First line contains N
- Second line contains N integers

Output Format

Print the sorted array.

Example

Input

5
89 45 78 12 99

Output

12 45 78 89 99

3) Arrange Exam Scores (Bubble Sort)

```
#include <stdio.h>
int main() {
    int N;
    scanf("%d", &N);
    int scores[N];
    for (int i = 0; i < N; i++) {
        scanf("%d", &scores[i]);
    }
    for (int i = 0; i < N-1; i++) {
        for (int j = 0; j < N-i-1; j++) {
            if (scores[j] > scores[j+1]) {
                int temp = scores[j];
                scores[j] = scores[j+1];
                scores[j+1] = temp;
            }
        }
    }
    for (int i = 0; i < N; i++) {
        printf("%d", scores[i]);
        if (i < N-1)
            printf(" ");
    }
    printf("\n");
    return 0;
}
```

Input : 5

89 45 78 12 99

Output : 12 45 78 89 99

OneCompiler

Upgrade

AI

C

Run

Main.c

+

```
1 #include <stdio.h>
2 int main() {
3     int N;
4     scanf("%d", &N);
5     int scores[N];
6     for (int i = 0; i < N; i++) {
7         scanf("%d", &scores[i]);
8     }
9     for (int i = 0; i < N - 1; i++) {
10        for (int j = 0; j < N - i - 1; j++) {
11            if (scores[j] > scores[j + 1]) {
12                int temp = scores[j];
13                scores[j] = scores[j + 1];
14                scores[j + 1] = temp;
15            }
16        }
17    }
18    for (int i = 0; i < N; i++) {
19        printf("%d", scores[i]);
20        if (i < N - 1)
21            printf(" ");
22    }
23    printf("\n");
24    return 0;
25 }
```

Console

I/O

AI Agent

2 ms

STDIN

5

89 45 78 12 99

Output:

12 45 78 89 99

4. Arrange Library Book Numbers (Insertion Sort)

Problem Statement

A librarian receives book serial numbers one by one and wants to place each new book in the correct position.

Write a program using **Insertion Sort** to arrange book numbers.

Input Format

- First line contains N
- Second line contains N integers

Output Format

Print sorted book numbers.

Example

Input

5
55 23 87 11 34

Output

11 23 34 55 87

4 → Arrange Library Book Numbers (Insertion Sort)

```
#include <stdio.h>
int main () {
    int N;
    scanf ("%d", &N);
    int books [N];
    for (int i = 0; i < N; i++) {
        scanf ("%d", &books [i]);
    }
    for (int i = 1; i < N; i++) {
        int Key = books [i];
        int j = i - 1;
        while (j >= 0 & books [j] > Key) {
            books [j + 1] = books [j];
            j--;
        }
        books [j + 1] = Key;
    }
    for (int i = 0; i < N; i++) {
        printf ("%d", books [i]);
        if (i < N - 1)
            printf (" ");
    }
    printf ("\n");
    return 0;
}
```

Input : 5

55 23 87 11 34

Output : 11 23 34 55 87

OneCompilerUpgrade

AI C Run

Save

Main.c

```
1 #include <stdio.h>
2 int main() {
3     int N;
4     scanf("%d", &N);
5     int books[N];
6     for (int i = 0; i < N; i++) {
7         scanf("%d", &books[i]);
8     }
9     for (int i = 1; i < N; i++) {
10        int key = books[i];
11        int j = i - 1;
12        while (j >= 0 && books[j] > key) {
13            books[j + 1] = books[j];
14            j--;
15        }
16        books[j + 1] = key;
17    }
18    for (int i = 0; i < N; i++) {
19        printf("%d", books[i]);
20        if (i < N - 1)
21            printf(" ");
22    }
23    printf("\n");
24    return 0;
25 }
```

Console

I/O

AI Agent

1 ms

STDIN

5

55 23 87 11 34

Output:

11 23 34 55 87

5. Rank Players by Score (Selection Sort)

Problem Statement

A gaming club wants to rank players based on scores from lowest to highest.

Write a program using **Selection Sort**.

Input Format

- First line contains N
- Second line contains player scores

Output Format

Print sorted scores.

Example

Input

```
5
70 20 90 10 40
```

Output

```
10 20 40 70 90
```

5 ⇒ Rank Players by Score (Selection Sort)

```
#include <stdio.h>
```

```
int main() {
```

```
    int N;
```

```
    scanf("%d", &N);
```

```
    int scores[N];
```

```
    for (int i = 0; i < N; i++) {
```

```
        scanf("%d", &scores[i]);
```

```
    }
```

```
    for (int i = 0; i < N-1; i++) {
```

```
        int min_idx = i;
```

```
        for (int j = i+1; j < N; j++) {
```

```
            if (scores[j] < scores[min_idx]) {
```

```
                min_idx = j;
```

```
            }
```

```
        }
```

```
        int temp = scores[min_idx];
```

```
        scores[min_idx] = scores[i];
```

```
        scores[i] = temp;
```

```
    }
```

```
    for (int i = 0; i < N; i++) {
```

```
        printf("%d", scores[i]);
```

```
        if (i < N-1)
```

```
            printf(" ");
```

```
    }
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```

Input : 5

70 20 90 10 40

Output : 10 20 40 70 90

OneCompiler

Upgrade

AI

C

Run

Save

2 ms

Main.c

```
1 #include <stdio.h>
2 int main() {
3     int N;
4     scanf("%d", &N);
5     int scores[N];
6     for (int i = 0; i < N; i++) {
7         scanf("%d", &scores[i]);
8     }
9     for (int i = 0; i < N - 1; i++) {
10        int min_idx = i;
11        for (int j = i + 1; j < N; j++) {
12            if (scores[j] < scores[min_idx]) {
13                min_idx = j;
14            }
15        }
16        int temp = scores[min_idx];
17        scores[min_idx] = scores[i];
18        scores[i] = temp;
19    }
20    for (int i = 0; i < N; i++) {
21        printf("%d", scores[i]);
22        if (i < N - 1)
23            printf(" ");
24    }
25    printf("\n");
26    return 0;
27 }
```

Console

I/O

AI Agent

STDIN

5

70 20 90 10 40

Output:

10 20 40 70 90